

Rhilo Sotto
4/9/2025
COMPE 470L

Lab 6 Report

Introduction

In this lab, we designed and implemented a digital clock using Verilog and a 4-digit seven-segment display on an FPGA board. The clock displays hours (1–12) and minutes (00–59), with a visual update every second to simulate minute changes for demonstration purposes. The system supports three operation modes: a standard clock mode, an alarm setting mode using push buttons, and an alarm-enabled mode where an LED indicates when the current time matches the alarm time. We used BCD conversion, multiplexed display control, and mode switching from previous labs.

Verilog Code

Top Module

```
module top(  
    input CLK100MHZ, // 100MHz system clock  
    input btnC, // Center button for reset  
    input btnL, // Left button for minutes  
    input btnR, // Right button for hours  
    input [1:0] mode, // Mode  
    output [3:0] digit, // Digit select  
    output [6:0] seg, // Segment control  
    output [0:0] LED // Alarm LED  
);  
  
    // Internal signals  
    wire RST = btnC;  
    wire [3:0] hour_ten, hour_one, min_ten, min_one;  
    wire [3:0] d_hour_ten, d_hour_one, d_min_ten, d_min_one;  
    wire alarm;  
    wire db_btnL;  
    wire db_btnR;  
  
    // Clock control instance
```

```

clockControl clock_ctrl(
    .CLK(CLK100MHZ),
    .RST(RST),
    .mode(mode),
    .btn_L(db_btnL),
    .btn_R(db_btnR),
    .alarm(alarm),
    .hour_ten(hour_ten),
    .hour_one(hour_one),
    .min_ten(min_ten),
    .min_one(min_one),
    .d_hour_ten(d_hour_ten),
    .d_hour_one(d_hour_one),
    .d_min_ten(d_min_ten),
    .d_min_one(d_min_one)
);

buttonDebounce btn_L(
    .CLK(CLK100MHZ),
    .button_in(btnL),
    .button_out(db_btnL)
);
buttonDebounce btn_R(
    .CLK(CLK100MHZ),
    .button_in(btnR),
    .button_out(db_btnR)
);

// Display control instance
displayControl display_ctrl(
    .CLK(CLK100MHZ),
    .RST(RST),
    .hour_ten(d_hour_ten),
    .hour_one(d_hour_one),
    .min_ten(d_min_ten),
    .min_one(d_min_one),
    .digit(digit),
    .seg(seg)
);

// Connect alarm to LED
assign LED[0] = alarm;
endmodule

```

Display Module

```
module displayControl(
    input CLK,
    input RST,
    input [3:0] hour_ten, hour_one, min_ten, min_one,
    output reg [3:0] digit,
    output reg [7:0] seg
);
    // Internal signals
    wire display_CLK;
    reg [1:0] digit_sel;
    reg [3:0] bcd_in;
    // Generate 1kHz clock for display multiplexing (100MHz/100_000 = 1kHz)
    clockDivider #(100_000) display_clockgen (
        .CLK(CLK),
        .RST(RST),
        .enable(1'b1),
        .CLK_out(display_CLK),
        .CLK_pulse() // Leave unconnected if not needed
    );
    // Digit selection counter

    always @(posedge display_CLK or posedge RST) begin
        if (RST)
            digit_sel <= 2'b00;
        else
            digit_sel <= digit_sel + 1;
    end

    // Digit multiplexing
    always @(*) begin
        case(digit_sel)
            2'b00: begin digit = 4'b0111; bcd_in = hour_ten; end
            2'b01: begin digit = 4'b1011; bcd_in = hour_one; end
            2'b10: begin digit = 4'b1101; bcd_in = min_ten; end
            2'b11: begin digit = 4'b1110; bcd_in = min_one; end
        endcase
    end

    // BCD to 7-segment decoder
    always @(*) begin
        case(bcd_in)
```

```

    4'h0: seg = 7'b1000000; // 0
    4'h1: seg = 7'b1111001; // 1
    4'h2: seg = 7'b0100100; // 2
    4'h3: seg = 7'b0110000; // 3
    4'h4: seg = 7'b0011001; // 4
    4'h5: seg = 7'b0010010; // 5
    4'h6: seg = 7'b0000010; // 6
    4'h7: seg = 7'b1111000; // 7
    4'h8: seg = 7'b0000000; // 8
    4'h9: seg = 7'b0010000; // 9
    default: seg = 7'b1111111; // All segments off
endcase
end
endmodule

```

Control Module

```

module clockControl(
    input CLK,
    input RST,
    input [1:0] mode, // Mode: 00 = Default, 01 = Alarm, 10 = Set Alarm, 11 = Default
    input btn_L, // Increment minutes in Set Alarm mode
    input btn_R, // Increment hours in Set Alarm mode
    output reg alarm, // Alarm LED (LED[0])
    output [3:0] hour_ten, hour_one, min_ten, min_one, // BCD outputs for current time

    output reg [3:0] d_hour_ten, d_hour_one, d_min_ten, d_min_one,
    input hour_inc, // Hour increment signal
    input min_inc // Minute increment signal
);

reg [3:0] a_hour_ten, a_hour_one, a_min_ten, a_min_one;

wire CLK_1hz;
wire min_overflow, hour_overflow;
reg [2:0] state;
reg [31:0] a_counter; // Counter to keep alarm LED on for 5 seconds
reg btn_L_prev, btn_R_prev;
wire btn_L_pressed, btn_R_pressed;
assign btn_L_pressed = btn_L && !btn_L_prev;
assign btn_R_pressed = btn_R && !btn_R_prev;

// Instantiate clock divider to generate 1Hz clock signal

```

```

clockDivider clock_divider(
    .CLK(CLK),
    .RST(RST),
    .enable(1'b1),
    .CLK_out(CLK_1hz),
    .CLK_pulse() // Leave unconnected if not needed
);

// Instantiate BCD Counters for minutes and hours
BCDCounter minutes(.CLK(CLK_1hz), .RST(RST), .en(1'b1), .ten(min_ten), .one(min_one),
    .min_OVF(min_overflow));
hoursBCDCounter hours(.CLK(min_overflow), .RST(RST), .en(1'b1), .ten(hour_ten),
    .one(hour_one));
// Overflow conditions
//assign min_overflow = (min_ten == 5 && min_one == 9);
assign hour_overflow = (hour_ten == 1 && hour_one == 2);

initial begin
    alarm <= 0;
    a_hour_ten <= 0;
    a_hour_one <= 0;
    a_min_ten <= 0;
    a_min_one <= 0;
    a_counter <= 0;
    state <= 3'b000;
    btn_L_prev <= 0;
    btn_R_prev <= 0;
end

// State machine based on modeitches `mode`
always @(posedge CLK or posedge RST) begin
    if (RST) begin
        alarm <= 0;
        a_hour_ten <= 0;
        a_hour_one <= 0;
        a_min_ten <= 0;
        a_min_one <= 0;
        a_counter <= 0;
        state <= 3'b000;
        btn_L_prev <= 0;
        btn_R_prev <= 0;
    end
    else begin
        btn_L_prev <= btn_L;
    end
end

```

```

btn_R_prev <= btn_R;
// Update state based on mode[1:0]
case (mode)
    2'b11: state <= 3'b000; // Default mode
    2'b01: state <= 3'b001; // Alarm mode
    2'b10, 2'b00: state <= 3'b010; // Set Alarm mode
endcase
case (state)
    3'b000: begin // Default mode (tracks time only)
        alarm <= 0;
        a_counter <= 0;
        d_hour_ten <= hour_ten;
        d_hour_one <= hour_one;
        d_min_ten <= min_ten;
        d_min_one <= min_one;
    end
    3'b001: begin // Alarm mode (tracks time, activates alarm if time matches)
        d_hour_ten <= hour_ten;
        d_hour_one <= hour_one;
        d_min_ten <= min_ten;
        d_min_one <= min_one;
        if ((hour_ten == a_hour_ten) && (hour_one == a_hour_one) &&
            (min_ten == a_min_ten) && (min_one == a_min_one)) begin
            if (a_counter < 500000000) begin
                alarm <= 1; // Turn on alarm LED
                a_counter <= a_counter + 1; // Count up to 5 seconds
            end
        else begin
            alarm <= 0; // Turn off alarm LED after 5 seconds
        end
    end
    else begin
        alarm <= 0;
        a_counter <= 0;
    end
end
    3'b010: begin // Set Alarm mode (increment alarm time)
        d_hour_ten <= a_hour_ten;
        d_hour_one <= a_hour_one;
        d_min_ten <= a_min_ten;
        d_min_one <= a_min_one;
        if (btn_L_pressed) begin // Increment minutes
            a_min_one <= (a_min_one == 9) ? 0 : a_min_one + 1;
            if (a_min_one == 9) a_min_ten <= (a_min_ten == 5) ? 0 : a_min_ten + 1;
        end
    end
end

```

```

end
if (btn_R_pressed) begin // Increment hours
    if (a_hour_ten == 1 && a_hour_one == 2) begin
        a_hour_one <= 1;
        a_hour_ten <= 0;
    end else begin
        a_hour_one <= (a_hour_one == 9) ? 0 : a_hour_one + 1;
        if (a_hour_one == 9)
            a_hour_ten <= (a_hour_ten == 1) ? 0 : a_hour_ten + 1;
        end
    end
    alarm <= 0; // Keep alarm off in Set Alarm mode
end
default: alarm <= 0;
endcase
end
end
endmodule

```

Hours BCD Module

```

module hoursBCDCounter(
    input CLK,
    input RST,
    input en, // Enable signal for counting
    output reg [3:0] ten, // Tens digit
    output reg [3:0] one // Ones digit
);

initial begin
    ten <= 1;
    one <= 2;
end

always @(posedge CLK or posedge RST) begin
    if (RST) begin
        ten <= 1;
        one <= 2;
    end
    else if (en) begin
        if (one == 9) begin
            one <= 0;
            ten <= ten + 1;
        end
    end
end

```

```

        else if ((one == 2) && (ten == 1)) begin
            one <= 1;
            ten <= 0;
        end
        else begin
            one <= one + 1;
        end
    end
end
endmodule

```

BCD Module

```

module BCDCounter( input CLK,
    input RST,
    input en, // Enable signal for counting
    output reg [3:0] ten, // Tens digit
    output reg [3:0] one, // Ones digit
    output reg min_OVF
);
    initial begin
        ten <= 5;
        one <= 9;
    end

    always @(posedge CLK or posedge RST) begin
        min_OVF = 0;
        if (RST) begin
            ten <= 0;
            one <= 0;
        end
        else if (en) begin
            if ((one == 9) && (ten == 5)) begin
                one <= 0;
                ten <= 0;
                min_OVF = 1;
            end
            else if (one == 9) begin
                one <= 0;
                ten <= ten + 1;
            end
            else begin

```



```

        one <= one + 1;
    end
end
end
endmodule

```

Clock Divider Module

```

module clockDivider #(
    parameter DIVIDER = 50_000_000, // Default for 1Hz from 100MHz
    parameter WIDTH = $clog2(DIVIDER) // Automatically calculate required counter width
)(
    input wire CLK, // Input clock
    input wire RST, // Active high reset
    input wire enable, // Enable signal for clock division
    output reg CLK_out, // Divided clock output
    output wire CLK_pulse // Single cycle pulse at division boundary
);
    // Counter register with calculated width
    reg [WIDTH-1:0] counter;
    // Terminal count detection
    wire terminal_count = (counter == (DIVIDER - 1));
    // Generate a single cycle pulse at the division boundary

    assign CLK_pulse = terminal_count & enable;
    // Counter logic
    always @(posedge CLK or posedge RST) begin
        if (RST) begin
            counter <= 0;
            CLK_out <= 0;
        end else if (enable) begin
            if (terminal_count) begin
                counter <= 0;
                CLK_out <= ~CLK_out;
            end else begin
                counter <= counter + 1'b1;
            end
        end
    end
    // Parameter validation
    initial begin
        if (DIVIDER < 2) begin
            $error("DIVIDER must be at least 2");
        end
    end
endmodule

```

```
$finish;  
end  
end  
Endmodule
```

Button Debounce Module

```
module buttonDebounce(  
    input CLK, // 100MHz clock  
    input button_in, // Raw button input  
    output reg button_out // Debounced output  
  
);  
    reg [15:0] counter = 0;  
    reg button_ff1 = 0;  
    reg button_ff2 = 0;  
    reg button_ff3 = 0;  
    // Double flop for metastability  
    always @(posedge CLK) begin  
        button_ff1 <= button_in;  
        button_ff2 <= button_ff1;  
        button_ff3 <= button_ff2;  
    end  
    // Debounce logic  
    always @(posedge CLK) begin  
        if (button_ff3 != button_ff2) begin // Button state changed  
            counter <= 0;  
            button_out <= button_out;  
        end else if (counter == 49999) begin // ~0.5ms debounce time  
            button_out <= button_ff2;  
            counter <= counter;  
        end else  
            counter <= counter + 1;  
        end  
endmodule
```

Constraint File

Clock signal

```
set_property PACKAGE_PIN W5 [get_ports CLK100MHZ]
set_property IOSTANDARD LVCMOS33 [get_ports CLK100MHZ]
create_clock -add -name sys_CLK_pin -period 10.00 -waveform {0 5} [get_ports CLK100MHZ]
```

Switches

```
set_property PACKAGE_PIN V17 [get_ports {mode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {mode[0]}]
set_property PACKAGE_PIN V16 [get_ports {mode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {mode[1]}]
```

LEDs

```
set_property PACKAGE_PIN U16 [get_ports {LED[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
```

#7 segment display

```
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
```

```
set_property PACKAGE_PIN U2 [get_ports {digit[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {digit[0]}]
set_property PACKAGE_PIN U4 [get_ports {digit[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {digit[1]}]
set_property PACKAGE_PIN V4 [get_ports {digit[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {digit[2]}]
set_property PACKAGE_PIN W4 [get_ports {digit[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {digit[3]}]
```

##Buttons

set_property PACKAGE_PIN U18 [get_ports btnC]

set_property IOSTANDARD LVCMOS33 [get_ports btnC]

set_property PACKAGE_PIN W19 [get_ports btnL]

set_property IOSTANDARD LVCMOS33 [get_ports btnL]

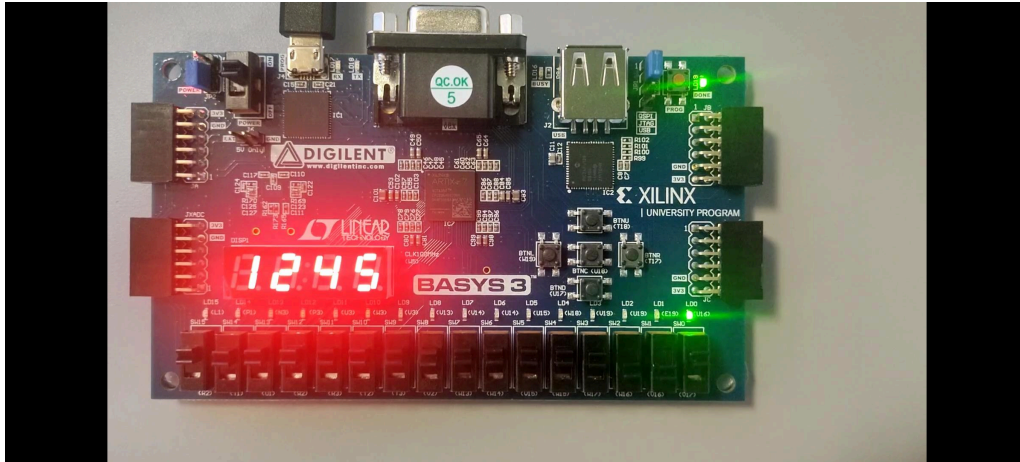
set_property PACKAGE_PIN T17 [get_ports btnR]

set_property IOSTANDARD LVCMOS33 [get_ports btnR]

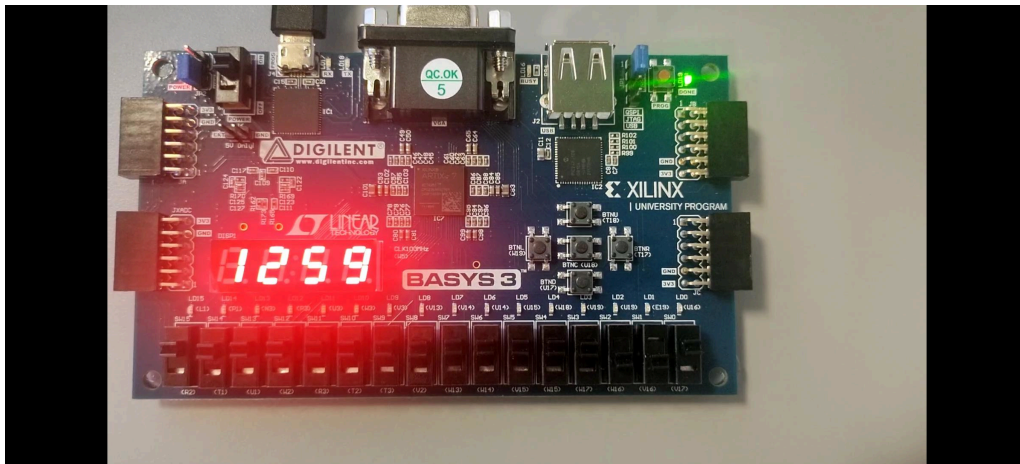
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]

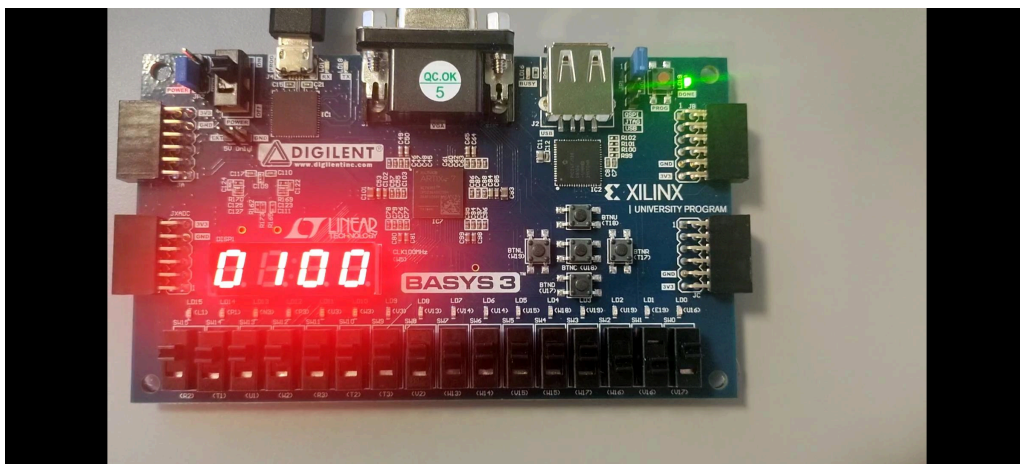
Results



Alarm LED on at 1245



Clock 1259



Clock at 0100 after 1259

Conclusion

In this lab, we successfully demonstrated the implementation of a multi-functional digital clock on an FPGA board. We utilized a combination of Verilog modules to control time counting, digit multiplexing, and mode switching. The clock functioned correctly across all three required modes, including alarm activation with visual LED feedback.